

Посібник з розгортання

Часова кореляція кіберзагроз — запустіть навчену мережу як рекомендаційний worker

Половина конвеєра, що відповідає за розгортання: візьміть мережу JNeopallium, яку створив тренер, під'єднайте її до джерел подій і приймача виводу та запустіть як безперервний worker безпеки в режимі «тінь → рекомендації» — налаштований саме так, як передбачає архітектура.

Документ	Посібник з промислового розгортання
Продукт	Модуль кібербезпеки Jнеорallium (Демо 06)
Споживає	Готову до розгортання мережу з Посібника з навчання
Режим безпеки	ADVISORY (лише рекомендації)
Платформи	Windows (PowerShell) і Linux/macOS (bash)
Автор	Дмитро Раковський — Харків, Україна
Дата	23 червня 2026
Ліцензія	BSD 3-Clause

Зміст

1. Чого ви досягнете
2. Передумови — навчений пакет і середовище виконання
3. Архітектура запуску
4. Готовий до розгортання пакет (що створило навчання)
5. Спрямуйте worker на згенеровану структуру нейромережі
6. Спакуйте JAR для розгортання
7. Конфігурація середовища: довідник властивостей IContext
8. Налаштуйте джерела подій і потоковий вхід
9. Під'єднайте агрегатор виводу та запустіть worker
10. Перехід від тіньового режиму до видимих оператору рекомендацій
11. Моніторинг у промислі
12. Безпека, відкат і пакування
13. Усунення несправностей
14. Додаток — шпаргалка розгортання

1 Чого ви досягнете

До кінця цього посібника ви візьмете **готову до розгортання мережу, створену Посібником з навчання** — теку конфігурації рівнів і нейронів JNeopallium зі справжніми класами часу виконання та вбудованими навченими вагами — і запустите її як безперервний **рекомендаційний worker**, що радить, але ніколи автономно не примушує до дій безпеки.

Стеля безпеки для всього посібника

Усе нижче працює в режимі ADVISORY. Worker може рекомендувати кандидатів на розслідування чи карантин, але не повинен ізолювати вузли чи блокувати трафік без окремого обґрунтування безпеки, процесу погодження та шляху відкату. Навчена мережа вже позначає рівень відповіді як ADVISORY, а жорсткий запобіжник — як фіксовану конфігурацію.

2 Передумови — навчений пакет і середовище виконання

- **Навчена мережа** з Посібника з навчання — тека артефактів із `model-descriptor.json`, `layer-0.json` ... `result-layer.json` і `trained-temporal-threat-model.json`.
- **Java 17+** для запуску worker і зібраний артефакт worker (`mvn clean install`).
- **Ваше джерело телеметрії** — шина подій у стилі Kafka (або будь-який потік, який можна адаптувати до типізованих сигналів).
- **Призначення виводу** — приймач JSONL для тіньового режиму або ваша SIEM / тема Kafka / вебхук для видимих оператору рекомендацій.

Спершу навчання

Якщо у вас ще немає навченої мережі, виконайте супровідний Посібник з навчання. Розгортання споживає його вихідну теку напряду — між навчанням і розгортанням немає кроку ручного перекладу.

3 Архітектура запуску

Промислове розгортання використовує справжній вхідний пункт worker — архітектура передбачає, що ви надаєте власний JAR, навчену мережу, конфігурацію та джерела подій. Worker запускається одним вхідним пунктом із чотирма аргументами:

```
java com.rakovpublic.jneuropallium.worker.application.Entry \
  <mode> <model-jar-url> <context-class> <context-json-or-path>
```

Аргумент	Значення	Приклад
mode	Топологія розгортання	local, http, grpc-client, grpc-master
model-jar-url	URL вашого JAR для розгортання	file:///opt/jneo/cyber-model.jar
context-class	Реалізація IContext	...runtime.DemoJsonContext
context-json-or-path	Вбудований JSON або шлях до файлу JSON	/opt/jneo/cyber-context.json

Усередині Entry передає чотири аргументи до Runner, який: (1) десеріалізує контекстний JSON у ваш клас IContext через Jackson; (2) завантажує JAR в ізолюваний завантажувач класів (JarClassLoaderService); (3) обирає застосунок за mode; і (4) запускає його. Режим прямо відображається на топології розгортання з архітектури:

Режим	Застосунок	Топологія
local	LocalApplication	Один JVM — пілоти, периферія
http	HttpClusterApplication	Розподілені робочі вузли через HTTP
grpc-master / grpc-client	GRPCServerApplication / GRPCCClient	Кластер gRPC; підтримка FPGA

Чотири промислові складові

Розгортання — це рівно чотири речі: (1) **навчена мережа** (з навчання), (2) **конфігурація IContext**, (3) **JAR для розгортання** (класи безпеки worker плюс ваше джерело подій і приймач) і (4) **ваші джерела подій та приймач виводу**. Наступні розділи з'єднують кожну з них.

4 Готовий до розгортання пакет (що створило навчання)

Тренер не видає абстрактну модель, яку треба перекладати, — він видає **власне мережу JNeopallium**. Пакет — це тека з десяти файлів; п'ять файлів рівнів плюс опис — це те, з чого завантажувється worker:

Артефакт	Роль під час розгортання
model-descriptor.json	Карта всієї мережі: 5 рівнів, 8 нейронів, мапа частот, класи часу виконання
layer-0.json	Межа багатоджерельного входу подій
layer-1-fast-evidence.json	Швидкі рецептори доказів (4 нейрони)
layer-2-temporal-correlation.json	Навчений корелятор — вбудовані ваги, гейти, політика базової лінії
layer-3-response-planning.json	Смуги рекомендаційної відповіді + фіксований жорсткий запобіжник
result-layer.json	Вивід рекомендацій безпеки
trained-temporal-threat-model.json	Компактна навчена модель, на яку посилається рівень кореляції
trained-model-update.json / quantitative-summary.json / source-mapping.json	Походження, метрики й відображення джерел

Уся мережа: **5 рівнів, 8 справжніх нейронів, 34 навчувані ваги, 1 зсув**. Кожен нейрон називає конкретний клас із пакета безпеки worker — тож розгортання зводиться до спрямування worker на цю теку, а не до повторної реалізації будь-чого.

5 Спрямуйте worker на згенеровану структуру нейромережі

Структура нейромережі — «файл структури нейронної мережі» з архітектури — **генерується навчанням**, а не пишеться вручну. Спрямуйте `configuration.input.layermeta` на теку з п'ятьма файлами рівнів, і worker завантажить мережу напряму. Кожен файл — це справжній рівень справжніх нейронів:

Файл рівня	Рівень	Справжні класи нейронів
layer-0.json	Вхід	(без об'єктів нейронів; лише канонічні входи)
layer-1-fast-evidence.json	Швидкі докази	NetworkFlowNeuron, SignaturePatternNeuron, ProcessBehaviourNeuron, EntityBehaviourBaselineNeuron
layer-2-temporal-correlation.json	Кореляція	TemporalThreatCorrelationNeuron
layer-3-response-planning.json	Планування + запобіжник	ResponsePlanningNeuron, ResponseGateNeuron
result-layer.json	Результат	ResponsePlanningNeuron (результат)

Рівень кореляції вбудовує все, що вивчила модель, готове до запуску:

```
"dendrites": {
  "weights": { "technique_command_and_control": 0.428,
               "network_receptor_score": 0.424,
               "maintenance_ratio": -0.452, ... },
  "bias": 2.1597, "decisionThreshold": 0.2 },
"baselineAdaptation": {
  "freezeWhenPosteriorAtLeast": 0.3,
  "freezeWhenSignatureConfidenceAtLeast": 0.8,
  "trustedBenignOnly": true }
```

Без кроку перекладу

Оскільки структуру згенеровано з уже вбудованими навченими вагами, те, що ви розгортаєте, — це саме те, що навчили й протестували. Поріг рішення (0.2), п'ять гейтів послідовності, смуги відповіді та пороги заморожування базової лінії — усе всередині файлів рівнів; нічого не вводиться вручну.

6 Спакуйте JAR для розгортання

Класи нейронів, процесорів і сигналів, на які посилається структура, **вже постачаються в модулі безпеки worker**, тож ви їх не реалізуєте повторно:

- Нейрони — `com.rakovpublic.jneuropallium.worker.net.neuron.impl.security.*` (TemporalThreatCorrelationNeuron, ResponseGateNeuron, ...).
- Процесори — `...worker.signalprocessor.impl.security.*` (AnomalyHypothesisProcessor, HypothesisResponseProcessor, QuarantineGateProcessor, ...).

- Сигнали — `...worker.net.signals.impl.security.*` (`AnomalyScoreSignal`, `ThreatHypothesisSignal`, `QuarantineRequestSignal`, ...).

Тож вашому JAR для розгортання потрібно надати лише дві інтеграційні частини, специфічні для вашого середовища: власне **джерело подій** (`IInitInput`, напр. читач `Kafka`) і власний **приймач виводу** (`IOutputAggregator`, напр. пересилач у SIEM). Зберіть його як модуль Maven, що залежить від артефакту `worker`:

```
mvn -pl cyber-deploy clean package
# -> target/cyber-model-1.0.jar (ваше джерело подій + приймач + worker на classpath)
# передається в Entry як: file:///opt/jneo/cyber-model.jar
```

Перевірка класів із швидкою помилкою

Перелічіть класи часу виконання, на які посилається ваша структура, у `configuration.neuronnet.classes` (тренер записує їх у `model-descriptor.json` під `generatedFrom.sourceRuntimeClasses`). Під час запуску `worker` перевіряє наявність кожного й переривається з ясною помилкою, якщо якогось бракує.

7 Конфігурація середовища: довідник властивостей IContext

Конфігурація доходить до `worker` через `IContext`. Вбудований `DemoJsonContext` читає JSON-документ із пласкою мапою `properties`; альтернативний клас `Context` читає класичний файл `.properties`. Обидва надають ті самі ключі через `getProperty(...)`. Властивості, які читає `worker`, зі значеннями, що визначають **розгорнуту рекомендаційну службу (висновок)**:

Властивість	Призначення / рекомендаційне значення
<code>configuration.input.layermeta</code>	Тека згенерованих файлів структури по рівнях
<code>configuration.neuronnet.classes</code>	CSV класів часу виконання, які мають бути в JAR (перевірка при старті)
<code>configuration.input.inputs</code>	Джерела подій — <code>InputArray</code> прив'язок <code>IInitInput</code>
<code>configuration.outputAggregator</code>	Клас <code>IOutputAggregator</code> , що отримує кожну рекомендацію
<code>configuration.storage.json</code>	Визначення <code>IStorage</code> (файловий стан для дамів нейронів / історії)
<code>configuration.processing.frequency.map</code>	Каданс {epoch, loop} кожного сигналу (з опису)
<code>configuration.slowfast.ratio</code>	Співвідношення швидкого до повільного циклу (N)
<code>configuration.history.slow.runs / .fast.runs</code>	Глибина історії сигналів для повільного / швидкого циклу
<code>configuration.isteacherstudying</code>	<code>`false`</code> для розгорнутого рекомендаційного <code>worker</code> ; <code>true</code> лише для навчання
<code>configuration.infiniteRun</code>	<code>`true`</code> для безперервної потокової служби
<code>configuration.maxRun</code>	Кількість запусків, коли не нескінченно (пакетне відтворення)
<code>configuration.runoncein</code>	Мілісекунди сну між циклами (каданс опитування)
<code>configuration.discriminatorsAmount</code>	<code>`0`</code> для звичайного рекомендаційного <code>worker</code>

Режим висновку проти режиму навчання

Установіть `configuration.isteacherstudying=false` з `discriminatorsAmount=0` та `infiniteRun=true`. Це переводить worker у безперервний рекомендаційний цикл: тягне сигнали з джерел подій → проганяє навчену мережу → записує рекомендації через агрегатор → повторює. Режим навчання (`true`) — для тренування, а не для живого детектора.

8 Налаштуйте джерела подій і потоковий вхід

Джерела подій — це місце, де промислова телеметрія входить у мережу.

`configuration.input.inputs` — це `InputArray` з одного чи кількох джерел; кожне прив'язує `IInitInput` (джерело) до `InputInitStrategy` (як його сигнали розміщуються в першому рівні):

```
{
  "inputData": [
    {
      "iInputSource": {
        "clazz": "com.acme.cyber.KafkaSecurityEventSource",
        "initInput": { "name": "cyber-stream", "epoch": 1, "loop": 1,
                      "topics": "auth,proc,dns,flow,intel" }
      },
      "mandatory": true,
      "initStrategy": {
        "clazz": "...signals.OneToAllFirstLayerInputStrategy",
        "iNeuronNetInput": {}
      },
      "amountOfRuns": 0
    }
  ]
}
```

Джерело подій реалізує контракт `IInitInput` — чотири методи: `readSignals()` повертає типізовані сигнали для цього циклу; `getName()`; `getDesiredResults()` (лише для навчання — порожній для живого детектора); `getDefaultProcessingFrequency()`.

Під'єднання справжнього джерела подій Kafka

Щоб споживати вашу живу телеметрію, реалізуйте `IInitInput`, що опитує ваші теми Kafka й перетворює кожен запис на типізовані сигнали безпеки, які споживає мережа — `PacketSignal`, `SyscallSignal`, `LogEventSignal` на рецепторах, що живлять `SignatureMatchSignal` і `AnomalyScoreSignal` у корелятор. Вкажіть його клас як `clazz` джерела. Зберігайте поле **часу події** кожної події, тож запізнiла, повторювана чи невпорядкована телеметрія все одно корелюється за часом події, точно як вимагає архітектура.

Безперервний потік проти пакетного відтворення

Для тривалої служби встановіть `configuration.infiniteRun=true` і `configuration.runoncein=<мс>` як інтервал опитування; worker зациклюється, тягнучи з джерела подій і видаючи рекомендації безперервно. Для обмеженого відтворення (наприклад, повторення інциденту) встановіть `infiniteRun=false` і `configuration.maxRun=<цикли>`.

Каданс кожного сигналу (типізовані часові масштаби)

`configuration.processing.frequency.map` призначає кожному сигналу його природний часовий масштаб через `{epoch, loop}`. Тренер записує рекомендований каданс в `signalFrequencyMap` опису — швидкі рецептори щоцикл, гіпотези й запити карантину кожен другий цикл, звіти про інциденти кожен п'ятий:

```
{
  "...security.PacketSignal":      { "epoch": "1", "loop": "1" },
  "...security.SyscallSignal":    { "epoch": "1", "loop": "1" },
  "...security.AnomalyScoreSignal": { "epoch": "1", "loop": "1" },
  "...security.ThreatHypothesisSignal": { "epoch": "1", "loop": "2" },
  "...security.IncidentReportSignal": { "epoch": "1", "loop": "5" }
}
```

9 Під'єднайте агрегатор виводу та запустіть worker

`configuration.outputAggregator` вказує `IOutputAggregator`. Його єдиний метод — `save(results, timestamp, run, context)` — викликається щоцикл із рекомендаціями, які видала мережа. Вбудований `JsonlResultAggregator` записує рекомендаційний JSONL, що ідеально як **приймач аудиту тіншового режиму**. Для промислу реалізуйте агрегатор, що пересилає рекомендації у вашу SIEM, тему Kafka чи вебхук, зберігаючи повний родовід доказів (сутність, діапазон часу події, джерела, апостеріорну, смугу відповіді, стан заморожування базової лінії, id і контрольну суму моделі).

Повний промисловий контекст поєднує все до купи — зверніть увагу, що `configuration.input.layermeta` вказує на згенеровану теку рівнів:

```
{
  "properties": {
    "configuration.input.layermeta": "/opt/jneo/cyber/layers",
    "configuration.neuronnet.classes": "...security.TemporalThreatCorrelationNeuron,...security.ResponsePlanningNeuron,...security.ResponseGateNeuron",
    "configuration.input.inputs": "{ ...InputArray з розділу 8... }",
    "configuration.processing.frequency.map": "{ ...мапа частот... }",
    "configuration.outputAggregator": "com.acme.cyber.SiemAdvisorySink",
    "configuration.storage.json": "{ \"storageClass\": \"...DemoFileStorage\", \"storage\": { \"rootPath\": \".../opt/jneo/cyber/state/\" } }",
    "configuration.slowfast.ratio": "10",
    "configuration.history.slow.runs": "8",
    "configuration.history.fast.runs": "64",
    "configuration.isteacherstudying": "false",
    "configuration.discriminatorsAmount": "0",
    "configuration.infiniteRun": "true",
    "configuration.runoncein": "1000",
    "configuration.maxRun": "0",
    "worker.threads.amount": "4"
  }
}
```

Запустіть worker, помістивши артефакт worker і його залежності на classpath і передавши ваш JAR для розгортання за URL (роздільник classpath: ; у Windows, : у Linux/macOS):

```
java -cp "worker-1.0.jar:libs/*" \
  com.rakovpublic.jneuropallium.worker.application.Entry \
```



```
local \
file:///opt/jneo/cyber-model.jar \
com.rakovpublic.jneuropallium.worker.demo.fullrun.runtime.DemoJsonContext \
/opt/jneo/cyber-context.json
```

Починайте з тіньового режиму

Спершу спрямуйте `configuration.outputAggregator` на приймач лише для аудиту (JSONL-агрегатор ідеальний), без сповіщення оператора. Worker транслює рекомендації, які ви можете порівнювати з тікетами інцидентів, перш ніж переходити до видимих оператору рекомендацій — тема наступного розділу.

10 Перехід від тіньового режиму до видимих оператору рекомендацій

Переводьте мережу поступово. Ніколи не переходьте одразу до видимого оператору виходу й ніколи не вмикайте примусове виконання без окремого погодженого проєкту.

1. **Розгорніть адаптери**, що перетворюють реальну телеметрію на типізовані сигнали безпеки.
2. **Завантажте мережу**, спрямувавши `configuration.input.layermeta` на згенеровану теку рівнів.
3. **Почніть у тіньовому режимі**: записуйте рекомендації в тему аудиту чи приймач JSONL без сповіщення операторів.
4. **Порівняйте** рекомендації з тікетами інцидентів, журналами red-team і вибірковою доброякісною активністю.
5. **Переведіть у видимий оператору рекомендаційний режим** лише після досягнення бюджетів хибних спрацювань і цілей затримки.
6. **Тримайте активне примусове виконання вимкненим**, доки не погоджено окремий проєкт керування відповіддю.

Справжній міст Kafka може замінити вхід демо. Зберігайте той самий типізований контракт подій і поля часу події, тож запізнiла, повторювана чи невідсортована телеметрія все одно корелюється за часом події, а не за часом обробки.

11 Моніторинг у промислі

Постійно відстежуйте ці метрики:

- Події, оброблені на джерело за хвилину; відкинуті, запізнiлі та невідсортовані події.
- Кількість рекомендацій на групу вузлів і орендаря; рівень перегляду хибних спрацювань; підтверджений рівень істинних позитивів.
- Середній час до виявлення та середній час до підтвердження оператором.
- Прийняті, заморожені та відхилені оновлення базової лінії; рівень придушення вікном обслуговування.
- Версія моделі та поріг, використані для кожної рекомендації.

Кожна рекомендація має зберігати родовід доказів: `entity_id`, діапазон `event_tick`, джерела й техніки, що долучилися, апостеріорну, смугу відповіді, `baselineFrozen`, `modelId` і контрольну суму моделі (`trainingChecksum` з `trained-model-update.json`).

12 Безпека, відкат і пакування

Пакування

Пакет розгортання — це згенерована тека артефактів плюс походження:

- Згенеровані файли рівнів і артефакти моделі (`model-descriptor.json`, `layer-0.json` ... `result-layer.json`, `trained-temporal-threat-model.json`, `trained-model-update.json`).
- SNA коміту Git, команда навчання, контрольна сума маніфесту, `trainingChecksum`, контрольні суми згенерованих артефактів, запис погодження рекомендаційного розгортання та ідентифікатор пакета відкату.

Правила безпеки та відкату

- Жорсткі запобіжники — це фіксована конфігурація в `layer-3-response-planning.json`, ніколи не навчені ваги моделі.
- Білі списки критичних активів і вікна обслуговування знижують терміновість, але ніколи не стирають докази.
- Адаптація базової лінії заморожується, коли апостеріорна сягає 0.3 або впевненість сигнатури сягає 0.8 — закодовано в блоці `baselineAdaptation` рівня.
- Тримайте попередній пакет мережі доступним для негайного відкату.
- Відкочуйте, якщо хибні спрацювання перевищують бюджет, покриття джерел падає або помилки розбору створюють систематичні сліпі зони.

Чесне обмеження

Вбудована еталонна мережа навчена на синтетичному детермінованому корпусі. Вона чудова для відтворюваних доказів конвеєра та регресії запобіжників, але не встановлює реальної якості виявлення. Промислові твердження потребують перенавчання на зовнішніх багатоджерельних наборах даних (див. Посібник з навчання) і валідації на репрезентативній корпоративній телеметрії.

13 Усунення несправностей

Симптом	Імовірна причина й виправлення
Cannot find class ... in provided jar	Бракує класу з <code>configuration.neuronnet.classes</code> — перезберіть JAR розгортання
Worker стартує, але нічого не видає	Перевірте <code>clazz</code> джерела подій і що <code>readSignals()</code> повертає типізовані сигнали
Рівень не завантажено / помилка структури	Переконайтеся, що <code>configuration.input.layermeta</code> вказує на згенеровану теку рівнів
Worker одразу завершується	Для живого детектора встановіть <code>infiniteRun=true</code> ; <code>maxRun</code> стосується лише обмеженого відтворення
Рекомендації видають блокувальну дію	Перевірте, що рівень відповіді <code>ADVISORY</code> ; це регресія

	для звіту
Неправильні поріг чи ваги під час роботи	Ви розгорнули застарілу теку рівнів; розгорніть найновіший навчений пакет
Проблеми з кирилицею / кодуванням у виводі	Переконайтеся, що агрегатор пише UTF-8

14 Додаток — шпаргалка розгортання

Запуск промислового рекомендаційного worker

```
java -cp "worker-1.0.jar:libs/*" \
  com.rakovpublic.jneuropallium.worker.application.Entry \
  local file:///opt/jneo/cyber-model.jar \
  com.rakovpublic.jneuropallium.worker.demo.fullrun.runtime.DemoJsonContext \
  /opt/jneo/cyber-context.json
```

Основи контексту рекомендаційного worker

```
configuration.input.layermeta      = <згенерована тека рівнів>
configuration.isteacherstudying    = false
configuration.discriminatorsAmount = 0
configuration.infiniteRun          = true
configuration.runoncein            = 1000
configuration.outputAggregator     = <ваш IOutputAggregator>
```

Модуль кібербезпеки Jнеорallium · Демо 06 · Посібник з розгортання. Споживає мережу, створену Посібником з навчання. Режим безпеки: ADVISORY. Активне примусове виконання потребує окремого погодженого обґрунтування безпеки. Ліцензія: BSD 3-Clause.